PROJECT SYNOPSIS

MediScan AI: Automated Clinical Laboratory PDF Parser & Risk Engine

1. Title of the Project

Title: MediScan AI: Automated Clinical Laboratory PDF Parser, Rule-Based Diagnostic Risk Engine & Generative AI Health Assistant

2. Introduction

Background: Clinical laboratory diagnostics form the foundation of modern healthcare, driving over 70% of medical decisions. However, standard lab reports are delivered as dense, unstructured PDF documents filled with complex medical terminology, raw numerical values, and reference ranges. Patients frequently struggle to interpret these parameters, leading to anxiety, delayed medical follow-ups, or misinterpretation of biological risk. **MediScan AI** addresses this gap by providing an end-to-end web platform that automatically ingests unstructured PDF lab reports, parses raw numerical biomarkers, evaluates parameters against clinical reference bounds, and translates diagnostic findings into visual health gauges, longitudinal trend graphs, and plain-language AI medical guidance.

Motivation: Traditional patient portals simply present PDF lab reports without contextual explanation. Existing digital solutions either rely exclusively on non-deterministic LLMs (which are prone to medical hallucinations or numerical errors) or rigid manual data entry (which is slow and error-prone). MediScan AI was undertaken to create a **Hybrid Medical Intelligence System** that combines the 100% mathematical precision of a deterministic clinical rule engine with the natural language summarization capabilities of Large Language Models (LLMs), giving patients clear, reliable, and actionable health insights.

3. Problem Statement

Description of the Problem: Laboratory PDF reports lack standardized formatting across diagnostic centers, often utilizing non-standard embedded font glyphs (Private Use Area - PUA encodings) or flattened image scans that prevent copy-pasting and standard text extraction. Furthermore, reference intervals vary dynamically based on patient age and biological sex. Patients reading raw lab values cannot easily assess cumulative health risk across multiple abnormal parameters or track long-term parameter trends over time. Existing AI solutions often send raw PDF text directly to LLMs, resulting in unverified numeric extractions, high token consumption, and potential hallucination of critical lab numbers.

4. Objectives (MAX 3)

Primary Objectives: *(Max 2)*

I. **Automated Hybrid PDF Extraction & Parameter Parsing:** Develop a robust, dual-mode OCR processing pipeline (combining PyMuPDF, custom PUA font decoders, and Tesseract OCR) capable of extracting raw text, medical parameters, units, and reference ranges from both digital and scanned laboratory PDFs with >95% accuracy.

II. **Deterministic Diagnostic Evaluation & Health Risk Engine:** Implement a rule-based clinical analysis engine and a 100-point weighted health scoring algorithm (health_risk_calculator.py) that evaluates parameters against age- and gender-tuned reference intervals without relying on non-deterministic AI for calculations.

Secondary Objectives: *(Max 2)*

I. **Longitudinal Health Tracking & Interactive Visual Analytics:** Build an interactive frontend interface featuring a 100-point radial health score gauge, historical trend charts, Health Icons parameter exploration, and automated branded PDF report generation.

II. **Context-Aware AI Assistant & Patient Management Suite:** Integrate a conversational Groq-powered Llama 3.3 70B AI assistant for plain-language medical guidance, alongside medication tracking, reminder scheduling, and PostgreSQL Row-Level Security (RLS) data isolation.

5. Scope of the Project

Scope:

• **What is Included:** Automated extraction and parsing of 20+ core clinical biomarkers (Hemoglobin, Fasting Glucose, HbA1c, Cholesterol, HDL, LDL, Triglycerides, Serum Creatinine, BUN, ALT, AST, TSH, Vitamin D, WBC, RBC, Platelets, CRP). Demographics tuning (Age & Gender) for clinical reference ranges. Deterministic 100-point Health Score and Risk Category assignment. Multi-report longitudinal trend charting and comparison. Conversational AI Health Assistant with full patient lab context. PDF export, follow-up scheduling, medication reminder management, and dark/light theme options.

• **What is NOT Included:** Direct integration with hospital Electronic Health Record (EHR) systems via HL7/FHIR protocols; formal medical diagnosis or diagnostic prescription (functions strictly as a decision-support tool); support for non-English lab reports or handwritten notes.

6. Methodology

The system adopts a **Modular Hybrid Software Methodology** divided into three execution layers:

1. **Ingestion & Normalization Layer:** Uploaded PDFs are parsed via PyMuPDF. If custom font glyphs are detected, a Private Use Area (PUA) font mapping table converts glyph codes to standard ASCII text. If digital text stream extraction fails, Tesseract OCR processes image frames.

2. **Deterministic Processing & Scoring Layer:** Extracted text is fed to regular expression engines to isolate parameter names, measured values, and units. Values are evaluated against medical_kb.py bounds tuned for the user's age and biological sex. The health calculator computes penalty deductions starting from 100.0, incorporating BMI and age multipliers.

3. **Generative AI & Presentation Layer:** Structured biomarker outputs are passed to the Groq Cloud Llama 3.3 70B model to generate plain-language narratives, key findings, and physician questions, displayed on a React 18 frontend dashboard.

7. System Design

a. **Architecture:** Decoupled Client-Server REST Architecture. React 18 single-page application frontend built with Vite, TypeScript, TailwindCSS, and Radix UI. FastAPI Python ASGI backend server. Supabase PostgreSQL database with SQLAlchemy ORM, connection pooling (pool_size=3, max_overflow=5), and Row-Level Security (RLS) policies.

b. **Components:**

• **Frontend Modules:** Dashboard, Reports Archive, Trends Visualizer, Biomarkers Directory, AI Assistant Chat, Reminders & Follow-ups, Settings Manager.

• **Backend Modules:** pdf_processor.py (OCR), parser.py (Regex), rule_analysis_engine.py (Clinical Rules), health_risk_calculator.py (Scoring), groq_service.py (LLM Integration), pdf_generator.py (ReportLab Export).

8. Development Process

a. **Technology Stack:** React 18, TypeScript, Vite, TailwindCSS, Framer Motion, TanStack Query, Recharts, Lucide Icons, Health Icons SVGs, Python 3.11+, FastAPI, PyMuPDF, Tesseract OCR, ReportLab, Supabase PostgreSQL, SQLAlchemy 2.0.

b. **AI/ML Integration:** Groq Cloud API hosting **Llama 3.3 70B Versatile** (280+ tokens/sec) and Llama 3.1 8B. Custom Paid Model API adapter (OpenAI, Anthropic, or self-hosted vLLM). Role of AI is strictly isolated to plain-language communication and chat, preventing numerical hallucination.

9. Implementation Steps

a. **Phase 1: Requirements Analysis & Core Engine Development:** Configured FastAPI layout, database schemas (User, Profile, Report, Biomarker, Reminder, Medication). Implemented PyMuPDF extraction, custom PUA font decoding, and regular expression biomarker parsers. Developed rule_analysis_engine.py and health_risk_calculator.py.

b. **Phase 2: Coding (Development & Integration):** Built React 18 SPA with dark/light themes, radial SVG health gauges, and Recharts trend visualizers. Integrated Groq Llama 3.3 70B API. Implemented PDF export engine via ReportLab, Supabase connection pooling, RLS security policies, and complete unit/integration test suites.

10. Testing Methodology

Testing Approaches:

- **Unit & Integration Testing:** Python test_api.py suite validating endpoints, DB operations, OCR parsers, and health score math (100% passed).
- **Type Safety & Build Verification:** Continuous TypeScript validation via npx tsc --noEmit (0 errors) and Vite production bundling (npm run build passed in 19.31s).
- **User Acceptance Testing:** Verification of responsive UI layouts, dark/light theme contrast, laser scan animation clipping, and interactive chart tooltips.

11. Expected Outcomes

A fully operational web application capable of processing digital and scanned laboratory PDFs in under 5 seconds, achieving 100% hallucination-free biomarker extraction, deterministic 100-point health score computation, historical trend visualization, and instant plain-language AI patient summaries.

12. Deliverables

1. **Source Code Repository:** Complete frontend (React/Vite/TS) and backend (FastAPI/Python) codebases hosted on GitHub (MunaafN/MediScan-AI).
2. **Working Software Application:** Deployed web platform featuring all 7 functional workspaces.
3. **System Documentation:** Complete Technical Architecture Guide PDF (MediScan_AI_Complete_Architecture_Guide.pdf).
4. **Database Schema & DDL Scripts:** Supabase PostgreSQL DDL migration scripts and SQLAlchemy ORM models.

13. Impact

Anticipated Impact: Demystifies complex lab reports for non-medical users, reduces health anxiety through clear explanations, and empowers proactive health management. For healthcare systems, it streamlines physician-patient communication by equipping patients with informed questions prior to consultations.

14. Timeline & 15. Project Plan (Tabular Format)

Project Phase	Key Milestones & Deliverables	Duration	Status
Phase 1: Ingestion & OCR	PyMuPDF, PUA Decoders, Tesseract OCR, Regex Parsers	Weeks 1 – 3	**Completed**
Phase 2: Rule Engine & Math	medical_kb.py, Age/Gender Bounds, Health Score Algorithm	Weeks 4 – 6	**Completed**
Phase 3: Frontend UI	React 18 SPA, Radial Health Gauge, Recharts Trends	Weeks 7 – 9	**Completed**
Phase 4: AI & PDF Export	Groq Llama 3.3 70B Integration, ReportLab PDF Engine	Weeks 10 – 12	**Completed**
Phase 5: Security & Testing	Connection Pooling, Supabase RLS, End-to-End Testing	Weeks 13 – 14	**Completed**

16. Resources Required

17. **Hardware:** Development Workstation (Intel i5/i7, 16GB RAM, SSD), Cloud Hosting Server (Render / Railway / AWS EC2 instance).

18. **Software:** Windows 11 / Linux, Python 3.11+, Node.js v18+, Supabase PostgreSQL Instance, Groq Cloud API Account, Visual Studio Code / Antigravity IDE, Git.

19. References

20. Literature:

1. World Health Organization (WHO), *'Laboratory Quality Management System Handbook,'* WHO Guidelines, 2021.

2. Clinical & Laboratory Standards Institute (CLSI), *'Defining, Establishing, and Verifying Reference Intervals in the Clinical Laboratory,'* CLSI EP28-A3c, 2010.

3. Touvron, H. et al., *'Llama 2: Open Foundation and Fine-Tuned Chat Models,'* Meta AI Research, 2023.

21. Tools and Technologies:

• FastAPI Documentation: https://fastapi.tiangolo.com | React 18 & Vite: https://react.dev

• Groq Cloud AI: https://groq.com | Supabase PostgreSQL: https://supabase.com | Health Icons: https://healthicons.org

23. Additional Information

Supplementary Clinical Engine Formulas & System Health Metrics:

- **Deterministic Threshold Cutoffs:** Critical Low = Ref_min × 0.60 | Borderline Low = Ref_min × 0.90 | Borderline High = Ref_max × 1.15 | Critical High = Ref_max × 1.50
- **Health Score Formula:** Health Score = 100.0 - Sum(Penalties) * BMI_Factor * Age_Factor
- **Verified Build Metrics:** 100% Backend API Test Pass (test_api.py), 0 TypeScript Errors (npx tsc --noEmit), 19.31s Production Build Time.

MediScan AI Project Synopsis | Submitted for Final Year Project Review